1. System Architecture Overview

The system follows a **Monolithic Monorepo** pattern, separating concerns into shared packages and specialized applications while maintaining a single source of truth for data.

The "Islands" Strategy

- **Static Layer (Astro):** Handles SEO-critical and content-heavy pages like the Landing Page, Resource Repository, and Public Showcases.
- **Interactive Layer (React 19):** Hydrated only where complex state management is required, such as the Challenge Submission Engine and live Leaderboards.

---

2. The Monorepo Blueprint

Built with **Turborepo** and **pnpm** for workspace efficiency and lightning-fast caching.

Applications (/apps)

- **web**: The public-facing portal built with Astro. Focused on speed, accessibility, and zero-JS payloads for static content.
- **dashboard**: The student/admin application. Uses **Astro SSR** to serve personalized data through Prisma and React for the interactive UI.

Shared Packages (/packages)

- **database**: The core of the system. Contains the **Prisma Schema** and the generated Prisma Client.
- **ui**: A shared design system built with **Tailwind CSS 4** and **Shadcn/ui** components.
- **config**: Unified ESLint, Prettier, and TypeScript configurations to maintain code quality across the team.

---

3. The Technology Stack

Layer	Technology	Purpose
Orchestration	**Turborepo + pnpm**	Manages the monorepo, build caching, and dependencies.

Layer	Technology	Purpose
Framework	**Astro 6**	Serves as the meta-framework for both static and SSR routes.
UI Library	**React 19**	Powers "Islands" of interactivity within Astro pages.
ORM	**Prisma**	Provides type-safe database queries and automated migrations.
Database	**PostgreSQL (Supabase)**	Handles relational data storage (Students, Challenges, Events).
Auth	**Supabase Auth**	Manages role-based access using school emails.
Styling	**Tailwind CSS 4**	Utility-first CSS for a rapid, mobile-first design.

---

4. Data Flow & Security

- **End-to-End Type Safety:** Prisma models are exported from @forge/database and used in both the Astro frontend and the React components, ensuring that a Challenge object always has the correct fields (Title, Difficulty, Points).
- **Row Level Security (RLS):** Supabase RLS policies ensure that while Prisma has full access to the DB, the client-side can only access data belonging to the authenticated student.
- **API Routes:** Astro's server-side endpoints act as the bridge between the React frontend and the Prisma/Postgres backend for sensitive operations like submitting challenge code or awarding badges.

---

5. Deployment & DevOps

- **Frontend/Edge: Vercel** for hosting the Astro monorepo, utilizing Edge Functions for fast SSR.
- **Database: Supabase** for managed Postgres and object storage for student file uploads.
- **CI/CD: GitHub Actions** for automated type-checking, linting, and deployment previews.